Map-free Reactive Navigation for a Holonomic Robot That Never Turns

RB2301 Final Project Report · [TBD: group number, names, matriculation numbers]

1. Introduction

The course has three parts: a zig-zag corridor with static obstacles, a narrow door, and two hand-released pendulums before the end wall. Our controller is **purely reactive**: it keeps no map, plans no path and is given no goal coordinate. Twenty times a second it turns the latest LiDAR scan into obstacle points and decides what to do from those points alone.

The robot is holonomic, so it **never turns to navigate**: it slides in any direction while holding its starting heading. Its own frame therefore stays lined up with the course, and every decision is made in that frame. The Optitrack pose is only used as memory: where the robot has been, where a pendulum is, and how far a dash has gone.

All three graded runs on the real course were successful [TBD: times], including runs where obstacles were placed deliberately to trap the robot before the door. Figure 2 shows one of them.

2. Code explanation

2.1 Organisation and control loop

Everything is in one ROS 2 node, `obstacle_course_reactive.py`. Besides the given template (subscriptions, pose callbacks, `move_2D`) it has four parts: a parameter block (shared values first, then the few that differ between Gazebo and the real robot); `timer_callback`, the 20 Hz loop below; navigation helpers (`scan_to_points`, `free_distance`, `best_lane`, `all_lanes`, `best_direction`); and pendulum helpers (`seen_past`, `in_front`, `pendulum_step` and two frame conversions).

```
every 50 ms:
    points <- LiDAR scan as (x, y) in the robot frame
    if heading > 6 deg off the start heading: turn back on the spot until within 2 deg; return
    if pendulum_step() is driving the robot: return
    if the current direction has been followed for 10 cm, or is now blocked:
        evaluate the best lane of each of the 16 directions and score them
        keep the current direction unless another scores clearly higher
    slide along the chosen direction, shifting sideways onto its lane
```

2.2 Navigation

Collision checking. `free_distance(points, θ)` returns how far the robot can slide along θ before touching any obstacle point. It sweeps the robot's rectangle along θ in 1 cm steps, all evaluated at once with NumPy. The rectangle is grown by a 1 cm clearance on every side except the one the robot moves away from, so it can never deadlock against a wall it is leaving. Because the robot never rotates, the swept rectangle is exact.

Lane shifts. With 16 directions 22.5° apart and no turning, a gap rarely lies exactly on one of the robot's directions. `best_lane` therefore also tries starting positions shifted sideways by up to ±12 cm in 2 cm steps. A shift only counts if the robot can first slide that far, and it must buy more room than it costs:

```
value = f - abs(s)        # room gained minus the shift
```

While driving, the robot slides onto the chosen lane (at twice the remaining shift, at most 0.15 m/s) and limits its forward speed to the straight free distance, so it creeps past an edge instead of hitting it (Figure 3).

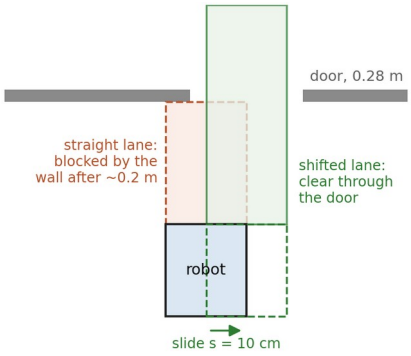


Figure 3. The straight lane hits the wall; sliding 10 cm sideways lines the robot up with the door.

Choosing a direction. A direction is *feasible* if its lane has at least 10 cm free (18 cm on the real robot, Section 3.5). Each feasible direction θ is scored by `best_direction` with four terms (Table 1):

```
score = goal_score + proximity_score + room_score + novelty_score
```

Table 1. The four terms of the direction score. Visited points are the robot's world positions, recorded every tick.

Term	How it is computed	What it does
Goal	cos(θ − 315°)	pulls towards the course's general direction (forward-right)
Room	1.0 × free distance along θ (up to 1 m)	prefers open directions; makes the way out of a pocket attractive
Novelty	1.8 × min(1, d / 0.4), d = distance from the spot it can reach (≤ 0.4 m along θ) to the nearest visited point	heads for unexplored space; zero for any direction into an already visited pocket
Time spent	−w × Σ (R − d)/R over visited points within R = 0.15 m of the spot 10 cm along θ (w = 0.25)	grows the longer the robot stays in one place, so it cannot stay trapped

Commitment and hysteresis. The robot follows a direction for 10 cm before choosing again (earlier if the path becomes blocked), and only evaluates all 16 directions at those moments. It then switches only if the best direction beats the current direction's *current* score by more than 0.1:

```
if not (self.selected_direction in self.feasible_directions
        and best_score <= scores[self.selected_direction] + self.hysteresis):
    self.selected_direction = best
```

2.3 Pendulums

Motion test. `seen_past` compares the newest scan with one taken 0.5 s earlier, after placing both in the world frame using their poses. For each point one scan hit, it finds the ray of the other scan pointing at that spot. If that ray and both its neighbours reach more than 10 cm further, the obstacle has moved:

```
for k in (-1, 0, 1):          # the ray and both neighbours must see past the point
    r = b[(idx + k) % n]
    past &= np.isfinite(r) & (r > dist + margin)
```

Run both ways, it finds things that left and things that arrived. A static wall never passes, however the robot moves, because both scans hit it.

State machine. `pendulum_step` runs before navigation every tick (Figure 1). When at least 10 moving points appear ahead, the nearest becomes the pendulum's *line*. Navigation keeps driving until the straight path to a stopping spot 10 cm before the line is clear (the robot is through the door). The robot then stands still for a 2 s check, creeps to the stopping spot and slides to about 6 cm from the right wall. There it waits until the pendulum has swung in front of it and away again: since a pendulum is slowest at the end of its swing, this leaves the longest gap before it returns.

```
if node.seen or node.clock - node.mode_start >= node.t_wait:    # passed and left, or timeout
    ahead = free_distance(node.occupied_points, 0.0, node.L, node.W)
    if ahead >= dash:                                           # only if the whole dash is clear
        node.mode, node.cross_start, node.cross_length = 'cross', node.pose[:2].copy(), dash
```

If the dash path is blocked (for example by a bent wall), the robot edges away from the wall until it is clear. After the second crossing it drives straight on until less than 5 cm is free: the goal.

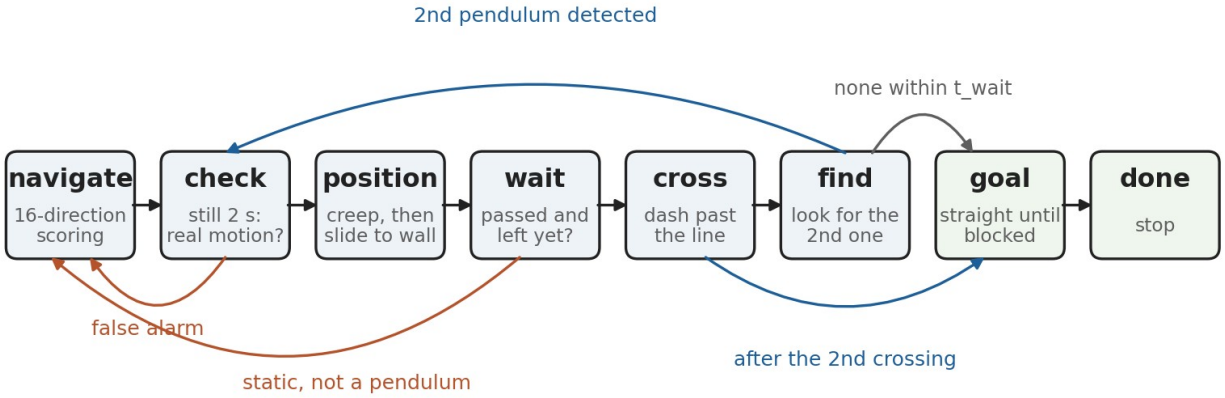


Figure 1. Pendulum state machine (pendulum_step). Orange arrows return to navigation when a detection turns out not to be a pendulum.

3. Results and observations on the real robot

3.1 Graded runs

All three graded runs reached the goal [VET: no contact with obstacles or pendulums]. Table 2 summarises them and Figure 2 shows one run.

Table 2. Graded runs on the real course (videos submitted separately).

Run	Course setup	Time	Notes
1	[TBD]	[TBD]	[TBD]
2	[TBD]	[TBD]	[TBD]
3	[TBD]	[TBD]	[TBD]

FIGURE 2 PLACEHOLDER

Composite of graded run [TBD]: frames ~1 s apart overlaid on one image, markers ①–⑦ (trap escape, door, check, wait at wall, dash, 2nd pendulum, goal)

Figure 2. Graded run [TBD]: frames about 1 s apart overlaid. ① escaping the trap, ② through the door, ③ 2 s check, ④ waiting at the wall, ⑤ dash, ⑥ second pendulum, ⑦ goal.

3.2 Always finding the door, even when trapped

[TBD: the professor's placements, e.g. obstacles on the right side of the middle corridor.] These placements target a weakness of goal-seeking robots: our goal direction (forward-right) pulls the robot into exactly those right-side pockets. Ours escapes for three complementary reasons:

- **Room** makes the open way out of a pocket attractive.
- **Novelty** drops to zero for every direction into a pocket after a single visit, so the robot does not go back in.
- **Time spent** keeps growing while the robot lingers, so it leaves even where everything nearby has been visited and novelty no longer helps.

Once out, **lane shifts** let it line up with the door from wherever it arrives, instead of having to arrive exactly aligned, the main difficulty for a robot that never turns (Figure 2, ①–②). In development tests (Table 3), adding novelty cut the slowest run from 57 s to 38 s, and lane shifts

cut the time through a 0.28 m gap, where the robot's centre has only a 9 cm band, from 62 s to 21 s.

3.3 Decisive motion: hysteresis

Without hysteresis, the robot flip-flopped between directions with similar scores, especially near walls, and moved back and forth. Two rules fixed it: commit to a direction for 10 cm, and switch only if another beats the current direction's **current** score by a margin. Comparing with the current score matters: the room term of the current direction shrinks as a wall gets close, so a stored score stays too high, and the robot drove on until blocked, then rebounded. Direction reversals (turns of 135° or more) fell from 3.8 to 0.4 per run in tests, and on the real course the robot moves in long, smooth segments [VET: video].

3.4 Reliable pendulum detection

Detecting motion from a moving robot is hard on real hardware. The LiDAR scan and the Optitrack pose arrive at different times, so whenever the robot changes speed or direction, static edges appear to shift and look like motion. Our detector removes these false alarms in layers, each simple and independent:

1. both-way test, needing the ray **and both neighbours** to see 10 cm past, which rejects noise and flickering edges;
2. only a window in front of the robot (0.15–1.0 m ahead, ±1.2 m sideways), so nothing beside it or outside the corridor ahead can trigger it;
3. at least 10 moving points to raise an alarm, since a real pendulum within 1 m produces dozens;
4. a 2 s stand-still confirmation needing 5 moving points at the line: with the robot still, timing errors vanish and only real motion survives;
5. memory of false spots, which never trigger again;
6. a crossing counts only if motion was seen at the line, so a false line can never start the final drive to the goal.

With 100 ms of scan delay in our 2D tests, the stand-still check alone raised successful runs from 17/21 to 21/21. On the real course, every pendulum in the graded runs was detected, confirmed and crossed [VET: video/terminal].

3.5 Built for the real hardware

Three key numbers come from the hardware rather than from tuning:

- **LiDAR blind below 15 cm.** The LiDAR is 4.5 cm ahead of the centre and the robot's half-length is 11.5 cm, so anything within about 8 cm of its front is invisible. The robot therefore waits 10 cm before a pendulum line, stops at 5 cm free before the end wall, and needs 8 cm of extra free space (`wall_gap`) in any direction it drives towards, so obstacles stay visible.
- **Heading-hold band.** The robot's heading drifts as it slides, and it is never placed perfectly straight. A tilted robot's "straight" dash along a wall converges on the wall, and its whole frame no longer matches the course. It therefore holds its start heading, turning on the spot at 0.2 rad/s. With an estimated 0.15 s of delay this overshoots by about 2°, so it starts correcting at 6° and stops at 2°: a band wider than the overshoot, so it cannot oscillate.
- **Detection thresholds.** 10 moving points to raise an alarm and 5 to confirm were set by experiment, from the gap between a real pendulum (dozens of points) and noise (a handful).

The Jetson Nano first ran the loop at only 5–6 Hz, because all 16 directions, with up to 15 shifted lanes each, were evaluated every tick. Evaluating them only when a direction is chosen, and only the current one in between, restored about 20 Hz with identical behaviour; the timers use real time, so they stay correct if a tick runs long.

Table 3. Effect of each feature in our 2D simulator (21 random layouts unless stated).

Feature	Without	With
Hysteresis on the current score	3.8 reversals per run	0.4 reversals per run
Novelty term	slowest run 57 s	slowest run 38 s
Lane shifts (0.28 m gap)	62 s to get through	21 s to get through
2 s stand-still check (100 ms scan delay)	17/21 runs through	21/21 runs through
Pendulum handling (252 randomised runs)	–	252/252 reached the goal, no obstacle hits

4. Limitations and future work

Pendulum timing margin. The robot goes once the pendulum has passed and turned away, and must then travel about 0.36 m (10 cm standoff + 23 cm robot + 3 cm pendulum) at 0.3 m/s, about 1.2 s plus start-up, to clear the swing plane. Modelling the swing as $x = A\cos(2\pi t/T)$, the pendulum re-enters the outer $d = 0.36$ m of its swing after $T - (T/\pi)\cdot\arccos(1 - d/A)$. With the measured period $T \approx 2$ s this is about 1.2 s for $A = 0.5$ m and 1.4 s for $A = 0.8$ m [VET: measure A]. The margin is thin on paper; it held in every graded run, but it should be designed in: estimate the period and phase from the motion seen during the 2 s check, start the dash at the moment that maximises the clear window, and skip a swing when the window is too short.

Unsynchronised scans and poses. The 2 s stand-still exists because each scan is paired with whatever pose has arrived, not with the pose at the scan's own time. Interpolating the pose to each scan's timestamp would make the motion test reliable while moving, removing the pause and allowing earlier detection from further away.

Fixed waiting side. The robot always waits at the right wall. A pendulum that swings only on that side never clears the robot's lane and forces a timeout crossing. The 2 s check already observes the swing; choosing the wall that the swing does not reach would remove this weakness.

Other limitations. Weights and thresholds are hand-tuned and could be tuned automatically on randomised layouts. Decisions are local and greedy: the time-spent penalty ensures escape in practice, not an efficient route. The LiDAR's blind zone forces margins that reject very tight gaps. And the no-turn design assumes the course never requires the robot to rotate.

5. Reflections

Simple is robust. Our first design maintained a global occupancy grid and planned A* paths through it [VET]. The map needed constant updating, noisy returns became phantom obstacles, the pendulums smeared across it, and it was hard to tell why the robot did anything. The reactive controller is simple in the sense that matters: every decision is local, explainable from the current scan, and uses only a handful of states. On the real course it was far more robust.

Measure before patching. On the real robot, the robot circled and raised frequent false alarms. We first adjusted weights and thresholds, which did not help, before checking what the robot believed: its Optitrack heading was about 175° off its true heading, which reversed every world-frame computation. Correcting that offset fixed both symptoms. Since then, we check the robot's beliefs (pose against direction of travel, loop rate, sensor limits) before changing any parameter.